

8장 성능 최적화 개념 정리

딥러닝에서 **성능 최적화**란 단순히 모델을 한 번 잘 학습시키는 것이 아니라, 데이터·알고리즘·하이퍼파라미터·하드웨어 등을 **종합적으로 조정하여** 모델의 **일반화 성능**을 최대화하는 과정을 의미한다.

8.1 성능 최적화 개요

성능 최적화는 크게 다음 네 가지 관점에서 접근할 수 있다.

1. 데이터를 활용한 성능 최적화
2. 알고리즘 선택을 통한 성능 최적화
3. 하이퍼파라미터 튜닝을 통한 성능 최적화
4. 앙상블을 이용한 성능 최적화

이 중 실제로 가장 많은 시간과 노력이 들어가는 부분은 **하이퍼파라미터 튜닝**이다.

8.1.1 데이터를 사용한 성능 최적화

(1) 데이터 양의 중요성

딥러닝과 머신러닝 모델은 **데이터의 양이 많을수록 성능이 향상되는 경향**이 있다. 이는 모델이 데이터로부터 패턴을 학습하는 구조이기 때문에, 데이터가 많을수록 더 일반적인 규칙을 학습할 수 있기 때문이다.

- 가능한 한 **많은 데이터를 수집**하는 것이 가장 기본적인 성능 향상 방법
 - 데이터가 부족한 경우:
 - 이미지 분야에서는 ****전이학습(Transfer Learning)****을 활용해 사전 학습된 모델의 지식을 재사용
-

(2) 데이터 분포 조정

신경망은 입력 데이터의 분포에 매우 민감하다. 따라서 데이터를 그대로 넣는 것보다 **분포를 조정**한 후 학습시키는 것이 중요하다.

- 시그모이드, 하이퍼볼릭 탄젠트 등의 활성화 함수는
입력 값의 범위에 따라 기울기 소멸 문제가 발생할 수 있음
 - 이를 방지하기 위해 **정규화, 표준화, 규제화** 기법을 사용
-

(3) 정규화 · 표준화 · 규제화 개념 구분

- **정규화(Normalization)**
 - 데이터 범위를 특정 구간(예: 0~1)으로 제한
 - 특성 스케일을 맞추는 과정 (feature scaling)
 - **표준화(Standardization)**
 - 평균 0, 표준편차 1이 되도록 변환
 - z-score 정규화라고도 함
 - 데이터가 가우시안 분포를 따를 때 효과적
 - **규제화(Regularization)**
 - 모델이 너무 복잡해지는 것을 막기 위한 제약
 - 과적합 방지를 목적
 - 드롭아웃, 조기 종료, 가중치 패널티 등
-

8.1.2 알고리즘을 이용한 성능 최적화

딥러닝에는 수많은 알고리즘과 모델 구조가 존재한다.

따라서 **처음 선택한 알고리즘이 최적일 아닐 가능성이 매우 높다.**

- 동일한 목적을 가진 여러 알고리즘을 비교 학습
- 검증 성능을 기준으로 **가장 성능이 좋은 알고리즘을 선택**
- 특정 알고리즘이 항상 정답은 아님

모델 선택 자체가 성능 최적화의 일부

8.1.3 알고리즘 튜닝을 위한 성능 최적화 (하이퍼파라미터 튜닝)

(1) 하이퍼파라미터 튜닝의 중요성

하이퍼파라미터 튜닝은

성능 최적화 과정 중 가장 많은 시간과 반복 실험이 필요한 단계

모델 하나를 선택한 후에도 다음 요소들을 계속 바꿔가며 실험해야 한다.

(2) 성능 진단(Diagnosis)

- 훈련 성능이 향상되다가 멈췄다면 **원인 분석 필요**
- 훈련 성능과 검증 성능의 차이를 통해 문제 파악

과적합 판단 기준

- train 성능 >> test 성능 → 과적합 의심
 - 해결 방법:
 - 규제화 기법 적용
 - 모델 단순화
 - 데이터 증가
-

(3) 주요 하이퍼파라미터 항목

① 가중치 초기값

- 일반적으로 **작은 난수**로 초기화
 - 더 나은 방법:
 - 오토인코더 등 비지도 학습을 이용한 사전 학습 후 지도 학습
-

② 학습률(Learning Rate)

- 네트워크 구조에 따라 적절한 값이 다름
 - 깊은 네트워크 → 상대적으로 큰 학습률
 - 얇은 네트워크 → 작은 학습률
 - 임의의 값으로 시작해 성능을 보며 점진적으로 조정
-

③ 활성화 함수

- 활성화 함수 변경 시 손실 함수도 함께 고려해야 함
 - 일반적 조합:
 - 은닉층: 시그모이드, 하이퍼볼릭 탄젠트, 렐루
 - 출력층: 소프트맥스, 시그모이드
-

④ 배치 크기와 에포크

- 최근 경향:
 - 작은 배치 + 큰 에포크
 - 하지만 데이터 크기에 따라:
 - 전체 데이터를 하나의 배치로 학습
 - 다양한 배치 크기 실험 필요
-

⑤ 옵티마이저와 손실 함수

- 기본: 확률적 경사 하강법(SGD)
 - 자주 사용:
 - Adam
 - RMSProp
 - 네트워크 구조에 따라 최적 조합이 다르므로 실험 필수
-

⑥ 네트워크 구성(Topology)

- 넓은 네트워크: 은닉층은 적고 뉴런 수가 많음
 - 깊은 네트워크: 계층 수는 많고 뉴런 수는 적음
 - 두 방식을 조합하여 최적 구조 탐색
-

8.1.4 앙상블을 이용한 성능 최적화

- 앙상블: 두 개 이상의 모델을 결합하여 사용
 - 단일 모델보다 일반화 성능 향상 가능
 - 단기간에 해결되지 않으며, 많은 실험과 시행착오 필요
-

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU의 차이

CPU

- 직렬 처리 방식
- 한 번에 하나의 명령어 처리
- 개별 코어 속도는 매우 빠름

GPU

- 병렬 처리에 최적화
 - 다수의 ALU를 이용해 동시에 연산
 - 개별 코어 속도는 CPU보다 느리지만 전체 연산량이 큼
-

8.2.2 GPU를 이용한 성능 최적화

- 대규모 행렬 연산을 동시에 처리 가능
 - CUDA 등장 이후 GPU 프로그래밍이 대중화
 - 딥러닝 학습 속도 대폭 향상
-

8.3 하이퍼파라미터 기반 성능 최적화 기법

8.3.1 배치 정규화(Batch Normalization)

(1) 배치 정규화의 필요성

신경망 학습 중 각 층을 지날 때마다 입력 데이터 분포가 변하는 현상 발생
→ 내부 공변량 변화

이로 인해:

- 기울기 소멸
- 기울기 폭발

문제가 발생

(2) 기울기 문제 정의

- 기울기 소멸
→ 역전파 중 기울기가 0에 가까워져 학습이 되지 않음
 - 기울기 폭발
→ 기울기가 급격히 커져 학습이 불안정해짐
-

(3) 배치 정규화 과정

1. 미니 배치 평균 계산
 2. 미니 배치 분산 및 표준편차 계산
 3. 정규화 수행
 4. 스케일 및 시프트로 분포 조정
-

(4) 장점과 단점

장점

- 학습 속도 향상
- 기울기 문제 완화
- 더 큰 학습률 사용 가능

단점

- 배치 크기가 작을 경우 불안정

- RNN 등 순차 모델에서는 구조 복잡

그럼에도 불구하고 성능 향상 효과가 커서 널리 사용됨

8.3.2 드롭아웃(Dropout)

(1) 과적합 현상

- 훈련 데이터 오류 ↓
 - 테스트 데이터 오류 ↑
-

(2) 드롭아웃 개념

- 학습 시 일부 뉴런을 무작위로 비활성화
 - 해당 뉴런의 가중치는 업데이트하지 않음
 - 매 학습 단계마다 다른 뉴런을 비활성화
-

(3) 효과

- 특정 뉴런에 대한 의존성 감소
 - 네트워크의 일반화 성능 향상
 - 지나친 학습 방지
-

(4) 테스트 시 처리 방식

- 모든 노드를 사용
 - 출력에 드롭아웃 비율을 곱하여 보정
-

8.3.3 조기 종료(Early Stopping)

(1) 개념

- 과적합을 방지하는 대표적 규제 기법

- 매 에포크마다 검증 데이터 손실을 모니터링
-

(2) 동작 원리

- 훈련 손실은 계속 감소
- 검증 손실이 증가하기 시작하는 시점에서 학습 종료
- 불필요한 학습을 막고 일반화 성능 유지